VIT BHOPAL

www.vitbhopal.ac.in

Project Report

STUDENT STRESS PREDICTOR

FUNDAMENTALS OF AI AND ML

Winter Semester:2025-2026

Name : Sancharika Ghosal.

Registration Number : 25BAI10963

2. Introduction

Student stress is a major academic issue influenced by lifestyle factors such as sleep, study duration, pending assignments, and emotional

wellbeing.

This project demonstrates how classical AI concepts (search strategies, knowledge representation, logic) can be combined with modern ML techniques to create an intelligent system that predicts stress levels (Low, Medium, High) through a hybrid rule-based + machine-learning approach

3. Problem Statement

College students often struggle with academic stress resulting from insufficient sleep, excessive study pressure, assignment load, and emotional fluctuations.

A lack of timely detection leads to poor performance, burnout, or anxiety.

The problem:

Can we build an AI-based system that predicts a student's stress level using basic lifestyle indicators?

Goal:

To analyze a student’s daily habits and predict whether their stress level is Low, Medium, or High using AI + ML techniques.

4. Functional Requirements

1. User Input Module
   - Takes sleep hours, study hours, number of pending assignments, and mood rating.
2. Rule-Based Reasoning Engine
   - Uses propositional logic and if–else rules to predict stress levels.
3. Uninformed Search (BFS) Module ◦ Traverses rule sets using Breadth-First Search to determine the most applicable rules.
4. ML Prediction Module
   - Trains a small Decision Tree Classifier for supervised learning.
5. Probability Module
   - Computes conditional probability based on sample dataset entries.
6. Final Output Module ◦ Combines rule-based output, ML output, and probability insights to produce a final stress prediction.

5. Non-functional Requirements

- Usability:

Simple console interface; beginner-friendly interaction.

- Reliability:

Handles invalid inputs and ensures consistent outputs.

- Maintainability:

Modular Python design allows easy modifications.

- Performance:

Executes all predictions instantly on basic hardware.

- Scalability:

System can integrate bigger datasets and better models in the future.

6. System Architecture

```
+----------------------+
|    User Input        |
+----------------------+
          |
v
+-----------------------------------+
|      Rule-Based Engine            |
| (Propositional Logic + BFS)       |
+-----------------------------------+
          |
v
+-----------------------------------+
|       ML Model (Decision Tree)    |
+-----------------------------------+
```

```
                    |
v
        +-----------------------------------+
        |    Probability Estimator          |
        +-----------------------------------+
                    |
v              Final
Stress Prediction
```

This hybrid architecture ensures:

- Explainability through rule-based logic
- Data-driven insights via machine learning
- Statistical insight via probability

7. Design Diagrams

A. Use Case Diagram

```
+-----------+
      |  Student  |
      +-----------+
            |
v
  [Enter Lifestyle Details]
            |
v
```

[Receive Stress Prediction]

B. Workflow Diagram Start

|

v

Input Student Data

|

v

Rul

e

En

gin

e

(Lo

gic

Ev

alu

ati

on)

|

v

BFS Search for Applicable Rule

```
  |
v
ML Model Prediction
  |
v
Probability Analysis
  |
v
Combined Stress Output
  |
v
End
```

C. Sequence Diagram

Student → InputModule → RuleEngine → BFS → MLModel → Probability → OutputModule → Student

D. Class / Component Diagram

```
+-------------------+     +-------------------+
| InputHandler      |     | RuleEngine        |
|-------------------|     |-------------------|
| get_user_input()  |     | apply_rules()     |
```

```
+------------------+     +------------------+


+------------------+     +------------------+
| ProbabilityModule |     | MLModel          |
|------------------|     | train()          |
| calc_prob()      |     | predict()        |
+------------------+     +------------------+


+-------------------------+
| MainController          |
|-------------------------|
| integrate_all_modules()  |
```

E. ER Diagram (Logical Data Model)

```
+------------+       +-----------------+
|   Student   | 1 --->| StressRecord     |
+------------+       +-----------------+
| sleep_hours |       | sleep            |
| study_hours |       | study            |
| mood        |       | mood_level       |
| assignments |       | assignments      |
+------------+       | stress_level     |
                     +-----------------+
```

8. Design Decisions & Rationale

- Decision Tree Classifier

Chosen for simplicity, interpretability, and suitability for small datasets.

- Propositional Logic Rules

Makes reasoning transparent and useful for viva + academic evaluation.

- BFS Search

Helps demonstrate uninformed search strategy from AI fundamentals.

- Hybrid Approach

Ensures the model uses both classical AI ideas and ML.

- Small Synthetic Dataset

Ideal for beginner-level courses; prevents unnecessary complexity.

9. Implementation Details 🎬

Implemented in Python 🎬 ML module uses scikit-learn 🎬 Probability computed using dictionarybased static dataset

🎬 Rule engine implemented using lambda functions and logic rules 🎬

BFS search implemented using collections.deque() 🎬 Modular structure improves readability

10. Screenshots / Results

10.1 Main Interface

Main Interface

Clean, minimalist interface with text input area and check button

```
========== Student Stress Prediction System ==========


--- Student Lifestyle Input ---
Hours of sleep (0 to 12):
```

```
========== Student Stress Prediction System ==========


--- Student Lifestyle Input ---
Hours of sleep (0 to 12):  8
Hours of study (0 to 12):  ↑↓ for history. Search history with c-↑/c-↓
```

```
========== Student Stress Prediction System ==========


--- Student Lifestyle Input ---
Hours of sleep (0 to 12):  8
Hours of study (0 to 12):  9
Assignments pending (0 to 10):  ↑↓ for history. Search history with c-↑/c-↓
```

```
========== Student Stress Prediction System ==========


--- Student Lifestyle Input ---
Hours of sleep (0 to 12):  8
Hours of study (0 to 12):  9
Assignments pending (0 to 10):  3
Mood level (1 to 5, where 1 = very low):  ↑↓ for history. Search history with c-↑/c-↓
```

10.2 Results Display

Results

```
========== Student Stress Prediction System ==========


--- Student Lifestyle Input ---
Hours of sleep (0 to 12):  8
Hours of study (0 to 12):  9
Assignments pending (0 to 10):  3
Mood level (1 to 5, where 1 = very low):  3

======= RESULTS =======
Rule-Based Prediction: LOW
Rule Matches Count: {'LOW': 0, 'MEDIUM': 0, 'HIGH': 0}

Machine Learning Prediction: MEDIUM

Probabilistic Insight:
P(Stress=HIGH | sleep<5) = 1.00

Final Recommendation:
Mixed signals → Final Level = MEDIUM (ML preferred)

==============================================
```

The program produces:

- Rule-Based Prediction: Using logical rules
- ML Prediction: Decision Tree classifier
- Probability Insight: Example: P(stress=HIGH | sleep < 5)
- Final Decision: Weighted combination of all three

The system correctly identifies stress patterns based on lifestyle data.

11. Testing Approach

Testing included:

- Functional Testing:
  Verified inputs, rule matches, ML predictions
- Boundary Testing: Examples: ◦ Very low sleep (2–3 hours) ◦ Very high assignments ◦ Mood = 1 or 5 extremes
- Negative Testing:
  Handled non-numeric inputs safely
- Cross-checking rule-based vs ML output consistency

12. Challenges Faced

- Designing rule-based logic without making it too complicated
- Ensuring BFS worked correctly with rule priorities
- ML model accuracy limitations due to small dataset
- Combining hybrid outputs in a meaningful way

.

13. Learnings & Key Takeaways

- ☐ Strong understanding of problem-solving methods and AI search strategies
- ☐ Practical application of knowledge representation

- Experience with probability, distributions, and conditional probability
- Exposure to a complete ML workflow: data → model → prediction
- Importance of modular code and clear documentation
- Realization of how classical AI and ML can complement each other

14. Future Enhancements

- Add a larger real dataset
- Use advanced ML algorithms like Random Forest
- Build a graphical user interface (GUI)
- Add stress trend monitoring
- Integrate with mobile or web applications
- Create personalized recommendations for students

15. References

- Stuart Russell & Peter Norvig — Artificial Intelligence: A Modern Approach
- Scikit-Learn Documentation
- Python Official Documentation
- Online articles on stress prediction models

End of Report